



Back To Basics

`std::unordered_map`

KEVIN CARPENTER



For decades, we taught C++ with
two containers:

`std::vector` and **`std::map`**

That's wrong.

At 10 million items,
map lookup is
~22x slower

Milliseconds become seconds...
seconds become awkward silence...
so you can buy your coffee.

So we all switched to
`std::unordered_map`.

What actually happens between
`m[key]` and the value?

Agenda

The Showdown	map vs unordered_map, why you care
Hashing	What a hash function actually is
Under the Hood	Buckets, nodes, and one insert at a time
Your Own Types	Hash, KeyEqual, and the rules they obey
Pitfalls[3]	Ordering, Collisions, Rehashing
Decision Tree	Your new framework

Round 1

map vs. unordered_map

How alike are they?

```
std::map<Mid, Auth> m;
```

```
std::unordered_map<Mid, Auth> m;
```

```
m["MID_100001"] = auth;           // insert, or overwrite
auto it = m.find("MID_100001");    // lookup
if (m.contains("MID_100001")) { ... } // C++20
m.erase("MID_100001");
for (const auto& [mid, auth] : m) { ... }
```

Every line below the declaration is **character for character identical**.

Same `value_type`: `std::pair<const Key, T>`. Same unique-key rule. Same allocator support.

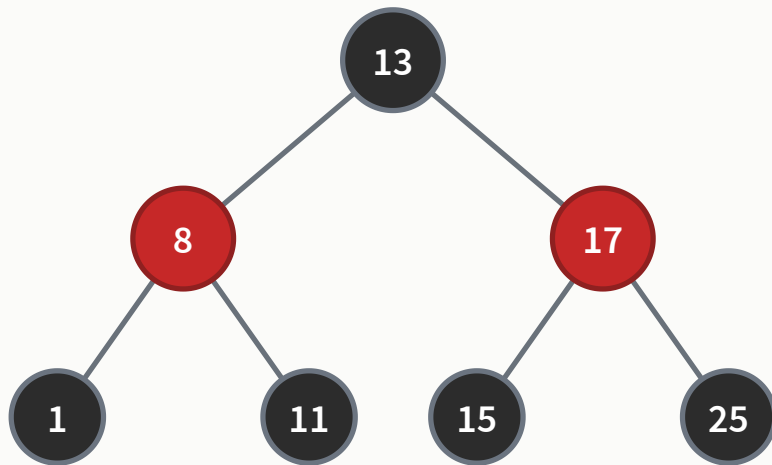
Both are **node-based**: one allocation per element, and references to elements stay valid across inserts.

One line of your program changes. That is exactly what makes the swap so easy to get wrong.

`std::map`: Red-Black Tree

- Guaranteed $O(\log n)$ lookup, insert, delete
- Keys always sorted
- Node-based: pointer-chasing, cache-unfriendly
- Stable iterators/references across all operations

What is a Red-Black Tree?



● red ● black nil sentinels not shown

Self-balancing $\rightarrow$ height stays $\approx \log n$

$\Rightarrow$ guaranteed $O(\log n)$
lookup / insert / erase

Every node is a separate
heap allocation

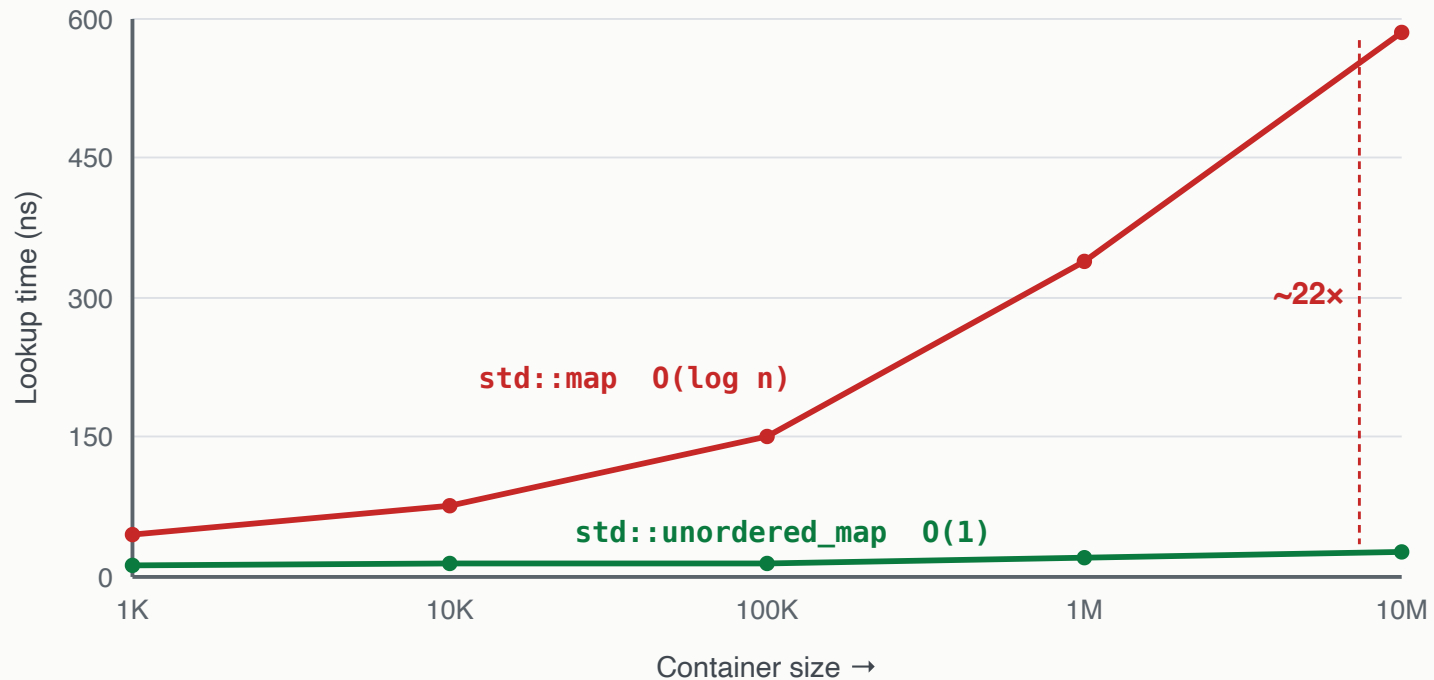
Coloring keeps the longest path $\leq 2 \times$ the shortest,
and that's the balance.

`std::unordered_map`

Hash Table

- Average $O(1)$ lookup, insert, delete
- Keys unordered
- Bucket-based: separate chaining (linked lists)
- *"Average"* is doing heavy lifting in that sentence

Lookup Time vs Container Size



Apple M5 Max / GCC 15.2 / -O2 · src/01_benchmark_map_vs_unordered_txn.cpp

What is a Hash?

No longer in the container's name.
Everything in its behaviour.

A hash function is a *bean grinder*

Any input, in. One fixed-size number, out.

Deterministic: same input, same output, every time *within one run*

Fixed size: a `std::string` of any length becomes one `size_t`

Well spread: similar inputs should give *completely different* outputs

It is **not** encryption. You can't get the key back.

It is **not** unique: infinite inputs, finite outputs. Collisions are guaranteed.

What `std::hash` actually is

```
std::unordered_map<std::string, AuthRecord> cache;
//          ^^^^^^^^^^^^^ hashed by std::hash<std::string>

template <class Key>
struct hash;                                // primary: DECLARED, never DEFINED

template <> struct hash<int>;                // ...but the standard
template <> struct hash<std::string>;        // ships these
```

A **class template**, not a function. Each specialisation is a **functor** with one `size_t operator()(const Key&)`. Make one, call it:
`std::hash<int>{}(42)`

Shipped for built-ins, pointers, `string`, `string_view`, `bitset`, smart pointers, `thread::id`, `optional`, `variant`, `filesystem::path`...

The primary template is undefined *on purpose*: ask for a type it doesn't cover and you get a **compile error**, never a silently bad hash.

There is none for `std::pair` or `std::tuple`. Remember that.

Hashing an `int`

```
std::hash<int> h;  
std::cout << h(0)      << '\n';    // 0  
std::cout << h(1)      << '\n';    // 1  
std::cout << h(42)     << '\n';    // 42  
std::cout << h(5000)   << '\n';    // 5000
```

That's the **identity function**.

`libstdc++`, `libc++` and `MSVC` all do this for small integers. It is not a bug.

Why it's fine: the map immediately does `hash % bucket_count()`. Sequential IDs land in sequential buckets, perfectly spread.

Hashing a `std::string`

```
std::hash<std::string> h;  
h("MID_100001");    // 1509000533495840331  
h("MID_100002");    // 17870333951025589230    <-- one character apart
```

Reads **every byte** of the string and mixes them together (MurmurHash2 on libstdc++).

One character different → a completely different number. That's the point.

It costs $O(k)$ in the key length k : hashing a 200-byte key reads 200 bytes.

src/03_under_the_hood_txn.cpp

What makes a hash *good*?

Property	What it means	If you get it wrong
Deterministic	same key → same bucket	you can never find anything
Uniform	keys spread across all buckets	one huge bucket → $O(n)$
Uses all the input	every field contributes	keys that differ collide anyway
Fast	you pay it on every operation	a perfect hash you can't afford

Note what is *not* on this list: **uniqueness**. You will never get it.

Under the Hood

How `std::unordered_map` really works

The Pipeline

Step 1: Key $\rightarrow$ `std::hash` $\rightarrow$ `size_t`

Step 2: Hash $\rightarrow$ `hash %`

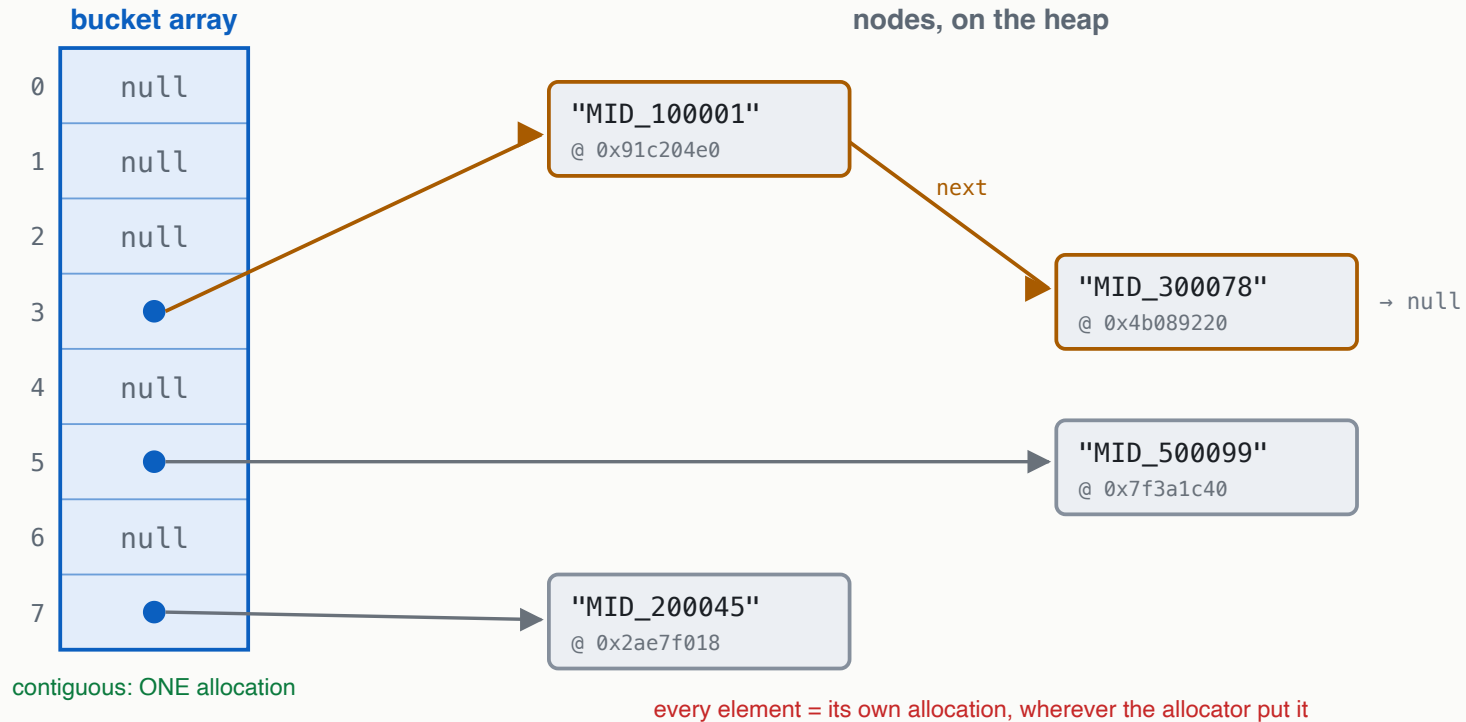
`bucket_count()` $\rightarrow$ Bucket Index

Step 3: Bucket $\rightarrow$ Linear scan $\rightarrow$ Data

```
size_t hash    = std::hash<std::string>{}("MID_100001"); // 15090005334
size_t bucket  = hash % merchant_cache.bucket_count();  // % 8 == 3
```

src/03_under_the_hood_txn.cpp

Anatomy of an unordered_map



bucket 3 holds two elements: that's a collision, chained

Insert #1: the happy path

```
merchant_cache["MID_100001"] = { "CPP Cache", 1 };
```

- 1 Hash the key: `std::hash<std::string>{} ("MID_100001")` $\rightarrow$ 1509000533495840331
- 2 Pick the bucket: $1509000533495840331 \% 8 \rightarrow$ bucket 3
- 3 Look in bucket 3. It's `null`. Nothing to compare against.
- 4 **Allocate a node** on the heap, move the pair into it
- 5 Point bucket 3 at the new node. Done.

Total cost: one hash, one modulo, **one allocation**, one pointer write.

Insert #2: the collision

```
merchant_cache["MID_300078"] = { "Gaylord Rockies", 7 }; // a diff
```

- 1 Hash it: 5112901386517366811. A *completely different* number.
- 2 5112901386517366811 % 8 → bucket 3. **Same bucket.**
- 3 Bucket 3 is occupied. Walk the chain and **compare keys**:
"MID_100001" == "MID_300078"? → no. Keep going.
Chain ends.
- 4 Allocate a node, link it into bucket 3's chain.

Two *different* keys, one bucket. Bucket 3 now costs 2 comparisons and 2 cache misses to search.

...and that's the whole story

1 item per bucket

hash $\rightarrow$ index $\rightarrow$ 1 compare

1 cache miss

$O(1)$

n items in one bucket

hash $\rightarrow$ index $\rightarrow n$ compares

n cache misses

$O(n)$

Same code. Same container.
The **hash** decides which one you get.

You can print that picture yourself

```
for (size_t b = 0; b < map.bucket_count(); ++b) {  
    if (map.bucket_size(b) == 0) continue;  
    std::cout << "Bucket " << b << " [" << map.bucket_size(b) << "]: ";  
    for (auto it = map.begin(b); it != map.end(b); ++it) // local iterator  
        std::cout << "'" << it->first << "\" ";  
    std::cout << '\n';  
}
```

```
Bucket count: 13          Load factor: 0.46  
  
Bucket 1 [1]: "MID_600023"  
Bucket 5 [1]: "MID_300078"  
Bucket 6 [2]: "MID_500099" "MID_400012"    <- a real collision  
Bucket 8 [1]: "MID_200045"  
Bucket 11 [1]: "MID_100001"
```

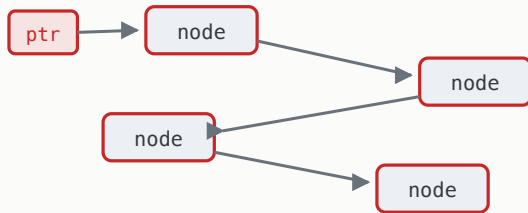
Not the 8 buckets from the diagram: libstdc++ opened at **13**, and the collision landed in a different slot. The *shape* is what transfers, never the specific numbers.

src/03_under_the_hood_txn.cpp, verbatim output

Node-based vs array-based

Node-based

map, unordered_map, list



4 elements → 4 allocations, scattered

Array-based

vector, flat_map, flat hash maps



one block, sequential

4 elements → 1 allocation, contiguous

One operator new per element

+ pointers & allocator bookkeeping per node

Elements never move → **stable addresses**

Traversal = pointer chase → **cache miss per hop**

One allocation, amortised over all of them

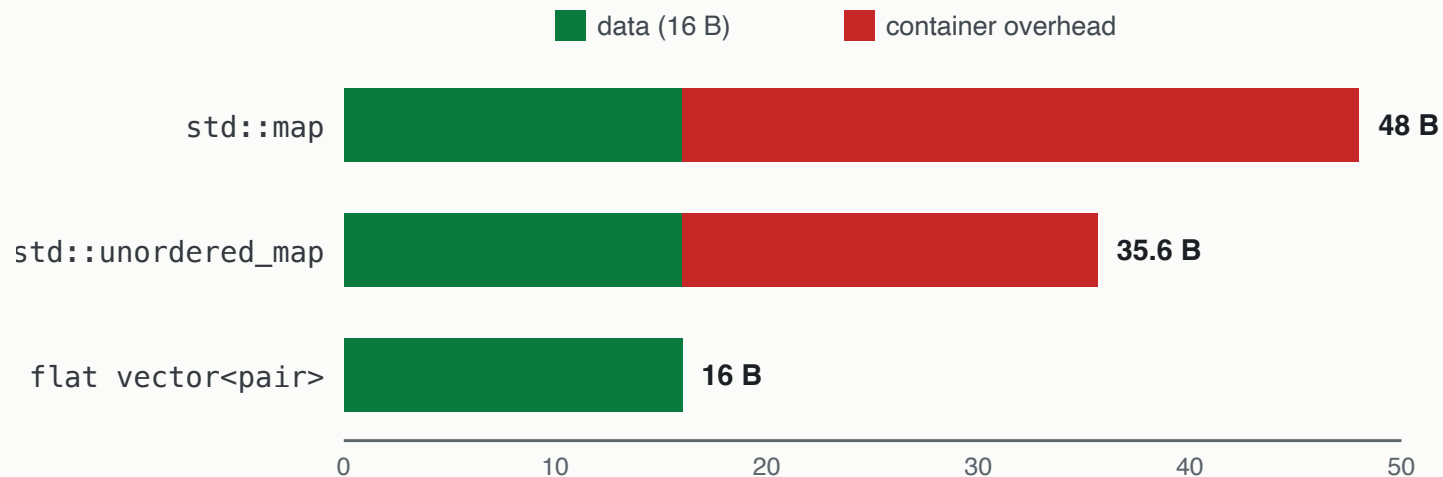
No per-element overhead

Growth *moves* elements → **addresses change**

Traversal = sequential → **prefetcher's dream**

The trade is always the same: **stable addresses** or **speed**. Pick one.

What a node actually costs



16 bytes of your data. `std::map` costs you 3x that;
`unordered_map` a little over 2x.

`src/14_memory_overhead.cpp`, counting allocator

Why not just use an array?

1. **The bucket interface.** `bucket_count()`, `bucket(key)`, `bucket_size(n)`, and local iterators `begin(n)/end(n)` that walk *one bucket*. That only means something if a bucket is a real, enumerable chain.
2. **Reference stability.** Insert and rehash must *not* invalidate pointers or references to existing elements. `&map[key]` has to stay valid across a grow.

Elements can never move → each one needs its own heap node → buckets hold pointers.

That is separate chaining. The standard never says the words, but nothing else conforms.

libstdc++, libc++ and MSVC all do exactly this.

$O(1)$ assumes each bucket has
~1 item.

$O(n)$ happens when one bucket has
ALL items.

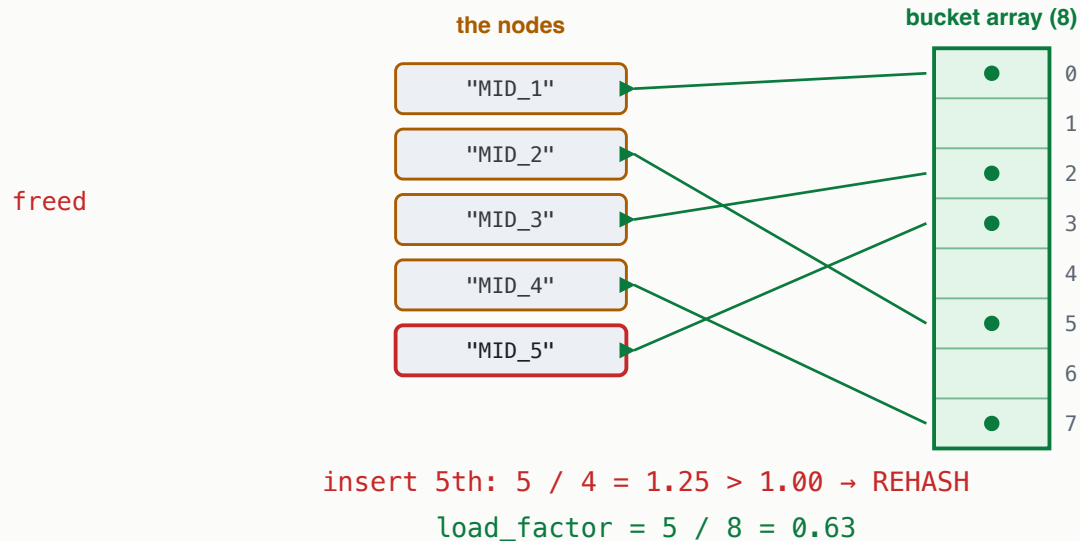
Load Factor

```
load_factor = size() / bucket_count()
```

When `load_factor > max_load_factor`
(default: **1.0**)

REHASH: Allocate new bucket array → re-hash
every key → re-link all nodes

What a rehash actually does



All of that happened **inside one insert: $O(n)$** in a call you expected to be **$O(1)$** .

The nodes never moved, so **references stay valid**, but every **iterator is now invalid**.

Your Own Types

Hash, KeyEqual, and the rules they obey

I want to key a map by *this*

```
struct Transaction {  
    std::string merchant_id;    // acquirer's merchant number  
    std::string card_bin;      // first 6 digits of the PAN  
    std::string stan;          // system trace audit number  
    uint32_t    amount_cents;  
    std::string currency;  
};  
  
std::unordered_map<Transaction, AuthResult> seen;    // ...does this compile?
```

Note what is *not* here: a timestamp. A retry carries the same STAN, so it must hash to the same bucket. **What goes in the key is a domain decision.**

src/04_pitfall1_bad_hash_txn.cpp

No. And the error is a wall.

```
error: use of deleted function  
      'std::hash<Transaction>::hash()'  
note:  'std::hash<Transaction>::hash()' is implicitly deleted  
      because the default definition would be ill-formed  
note:  ...500 more lines of template instantiation...
```

The standard library ships `std::hash` for the built-ins, `std::string`, pointers, `std::bitset`...

It cannot possibly ship one for **Transaction**. Only you know which fields identify it.

So you supply it. Two ways.

Two ways to supply a hash

1. Specialise `std::hash`

every map of this type gets it free

```
template <>
struct std::hash<Transaction> {
    size_t operator()(
        const Transaction& t) const {
        return hash_values(
            t.merchant_id, t.card_bin,
            t.stan, t.amount_cents,
            t.currency);
    }
};

// just works now:
std::unordered_map<
    Transaction, AuthResult> seen;
```

2. Pass a functor

different maps, different hashes

```
struct TransactionHash {
    size_t operator()(
        const Transaction& t) const {
        return hash_values(
            t.merchant_id, t.card_bin,
            t.stan, t.amount_cents,
            t.currency);
    }
};

// name it as a template arg:
std::unordered_map<
    Transaction, AuthResult,
    TransactionHash> seen;
```

Same function body. Option 1 for "this is *the* hash for my type"; option 2 when one type needs several.

`src/05_pitfall1_hash_combine_txn.cpp`

So what goes *in* the hash?

BAD

```
struct BadHash {  
    size_t operator()(  
        const Transaction& t) const {  
        return std::hash<std::string>{}(  
            t.merchant_id);  
        }  
};  
  
// "the merchant identifies it"  
// One anchor merchant = 80% of  
// your auths in ONE bucket.
```

GOOD

```
struct GoodHash {  
    size_t operator()(  
        const Transaction& t) const {  
        size_t seed = 0;  
        // Order-dependent mixing:  
        hash_combine(seed, t.merchant_id);  
        hash_combine(seed, t.card_bin);  
        hash_combine(seed, t.stan);  
        hash_combine(seed, t.amount_cents);  
        hash_combine(seed, t.currency);  
        return seed;  
    }  
};
```

Why not plain `seed ^= hash(field)`? XOR is commutative: swap merchant/BIN and you get the *same* hash.

src/04_pitfall1_bad_hash_txn.cpp

The Pro-Tip: hash_combine

```
template <typename T>
void hash_combine(size_t& seed, const T& val) {
    const size_t GOLDEN = 0x9e3779b9;
    seed ^= std::hash<T>{}(val) + GOLDEN
           + (seed << 6) + (seed >> 2);
}

template <typename... Args>
inline size_t hash_values(const Args&... args) {
    size_t seed = 0;
    (hash_combine(seed, args), ...); // C++17 fold expression
    return seed;
}
```

```
struct TransactionHash {
    size_t operator()(const Transaction& t) const {
        return hash_values(
            t.merchant_id, t.card_bin, t.stan,
            t.amount_cents, t.currency);
    }
};
```

src/05_pitfall1_hash_combine_txn.cpp

The parameters you never looked at

```
template<
    class Key,
    class T,
    class Hash      = std::hash<Key>,          // key -> size_t
    class KeyEqual  = std::equal_to<Key>,      // are these two keys the same?
    class Allocator = std::allocator<std::pair<const Key, T>>
> class unordered_map;
```

You have used parameters 1 and 2 for years.

You just met parameter 3.

Parameter 4 is the one that bites.

KeyEqual: hashing is only half the job

Remember insert #2, two different keys in one bucket. Something had to *compare* them.

Hash narrows you to *one bucket*. It is allowed to be wrong, because collisions are fine.

KeyEqual picks the right element *inside* that bucket. It is *never* allowed to be wrong.

Default is `std::equal_to<Key>`, which calls your `operator==`.

Write a custom hash for your struct and forget `operator==`?
Another wall of template errors. Hash and equality always ship together.

The one rule you must not break

$$a == b \Rightarrow \text{hash}(a) == \text{hash}(b)$$

If two keys are *equal*, they must hash to the *same bucket*.

Otherwise they land in different buckets and are **never compared**, so the map never notices they're the same key.

The reverse is *not* required: equal hashes need not be equal keys. That's just a collision.

Break it and nothing crashes. You simply **cannot find keys you just inserted**.

How you break it by accident

```
struct CaseInsensitiveEqual {
    bool operator()(const std::string& a, const std::string& b) const {
        return std::ranges::equal(a, b, [](char x, char y) {
            return std::tolower(x) == std::tolower(y); });
    }
};

std::unordered_map<std::string, int,
                  std::hash<std::string>, // case SENSITIVE
                  CaseInsensitiveEqual>    // case INsensitive
    card_types;

card_types["visa"] = 1;
card_types.find("VISA"); // end() -- but we said they're equal!
```

`hash("visa") != hash("VISA")` → different buckets →
`CaseInsensitiveEqual` is never called.

The fix: lower-case inside the hash too. Change one, change both.

So when do I have to care?

Your key is...	Hash	KeyEqual
<code>int</code> , <code>std::string</code> , pointer	free	free
your own struct	you write it	<code>operator==</code>
<code>std::pair</code> / <code>std::tuple</code>	you write it (no <code>std::hash</code> !)	free
an enum	free (since C++14)	free
"equal" in a custom sense	both, and they must agree	both

Row 3 catches people constantly:

```
std::unordered_map<std::pair<int,int>, V>  
does not compile.
```

When $O(1)$ Becomes $O(n)$



Pitfall 0: The Ordering Trap (**correctness**)

Pitfall 1: The Collision Catastrophe

Pitfall 2: The Hidden Rehash

Pitfall 0: It's in the name

The next two pitfalls cost you *performance*.

This one costs you **correctness**,
and it bites first.

A real auth service

A per-request `std::map` is iterated in order to build the wire message...

```
std::string requestMapToMML( std::map<std::string,std::string>& req ) {  
    std::string mml;  
    for ( const auto& field : req ) {           // sorted iteration  
        mml.append( toMmlTag(field.first) );  
        mml.append( field.second );  
        mml += MML_FIELD_SEP;  
    }  
    mml += Crypto::SHA1( mml );                 // order is now in the hash  
    return mml;  
}
```

Swap to `unordered_map` → fields reorder → bytes change →
SHA1 changes → the switch rejects the message.

distilled from a real payment-auth service · `src/22_ordering_is_load_bearing.cpp`

Same data, one swap

```
std::map          AMOUNT5000|AUTH_GUIDA1B2|BATCH_ID42|CARD_BIN41111|...  
                  hash = 6012151533857893158    ACCEPTED  
  
std::unordered_map BATCH_ID42|TRACEZZ9|TERM_IDTERM07|CURRENCYUSD|...  
                  hash = 6411950521819669982    REJECTED: wire protocol broken
```

No bad hash. No collision. No rehash. A pure
correctness bug.

Hides anywhere iteration order is load-bearing: wire formats, signatures/HMACs, golden-file tests, dedup keys.

Pitfall 1: hashing by card network

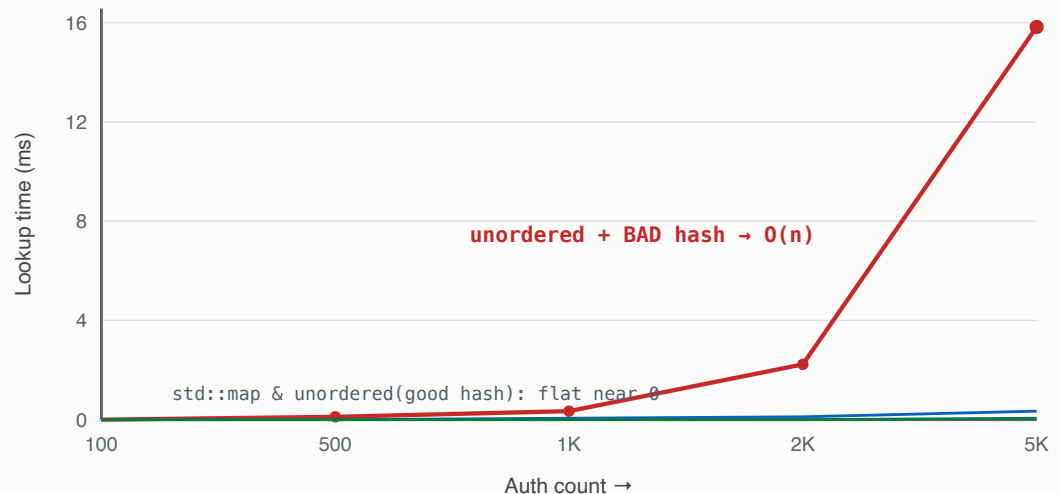
```
struct CardTypeHash {  
    size_t operator()(const std::string& auth) const {  
        return std::hash<std::string>{}(  
            get_card_type(auth));    // VISA, MC, AMEX, DISC  
        }  
};
```

At 5,000 auths:

~305x slower

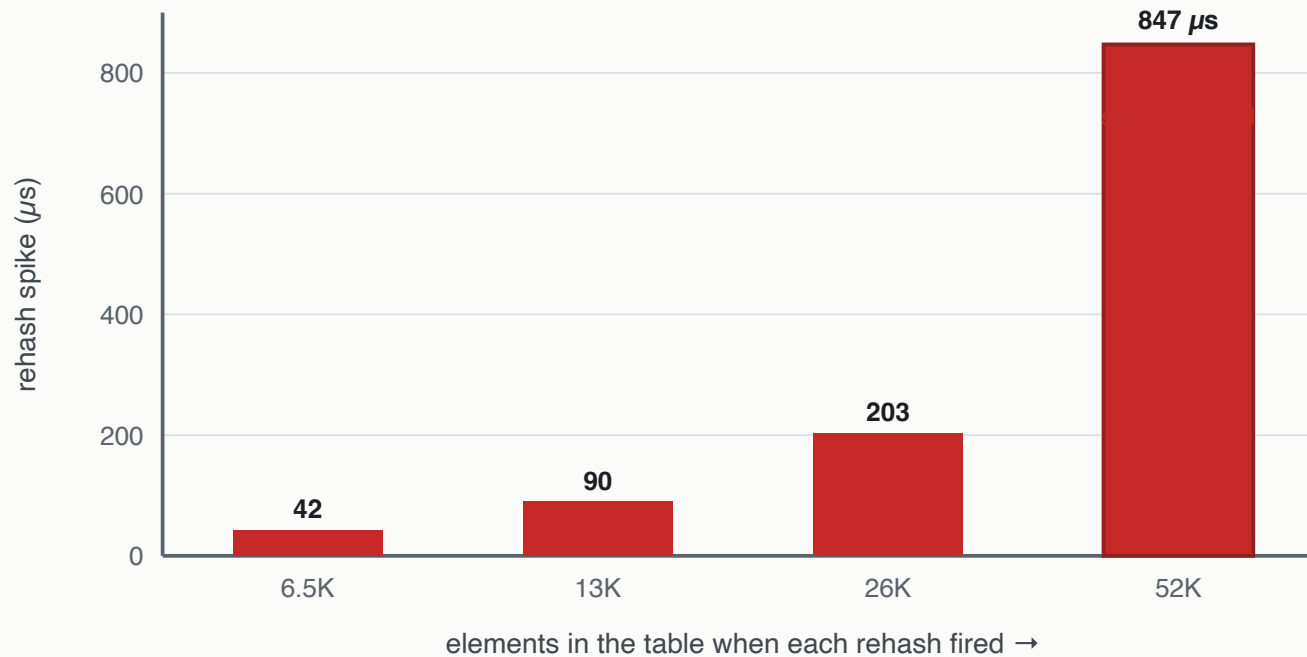
vs a good hash: 15.8 vs 0.05 ms

~47x slower than `std::map`



src/06_pitfall2_collision_catastrophe_txn.cpp · M5 Max / GCC 15.2

Pitfall 2: The Hidden Rehash



src/07_pitfall3_rehash_txn.cpp

The Fix: `reserve()`

	Without <code>reserve()</code>	With <code>reserve()</code>
Avg insert time	0.42 us	0.31 us
Max insert time	2847.00 us	8.21 us
Rehash events	21	0

```
std::unordered_map<std::string, uint32_t> auth_cache;  
auth_cache.reserve(expected_auth_count);    // one line, zero rehashes
```

src/08_pitfall3_reserve_fix_txn.cpp

Beyond unordered_map

C++23 & Modern Alternatives

std::flat_map (C++23)

```
#include <flat_map>

using BinTable_t = std::flat_map<
    std::string, BinInfo>;
BinTable_t bin_table;
bin_table["411111"] = {"CHASE", "VISA"};
auto it = bin_table.find("411111");
```

- Backed by two sorted `std::vectors` (contiguous memory)
- Cache-friendly iteration: no pointer chasing, so read-heavy workloads fly
- Trade-off: $O(n)$ insert, but BIN tables load *once* at startup
- Think: `std::map` meets `std::vector`

src/09_cpp23_flat_map_txn.cpp

Iterator Stability: When to Use `std::map`

```
std::unordered_map<std::string, AuthRecord> auth_cache;  
auth_cache["TXN_0001"] = {"AUTH_A1B2", 5000};  
  
auto pending = auth_cache.find("TXN_0001"); // reversal handler  
  
for (int i = 3; i < 20; ++i) // rehash triggers...  
    auth_cache["TXN_" + std::to_string(i)] = { /* ... */ };  
  
// pending is NOW INVALID: UB to dereference!
```

`unordered_map`: rehash invalidates **iterators**
(pointers/references survive)

`std::map`: insertion **NEVER** invalidates iterators, pointers, or references

The Decision Framework

Choosing the Right Container

What you now know

A hash is a **meat grinder**: deterministic, fixed-size, well spread. Never unique.

The map does **hash** → **modulo** → **scan the bucket**. That's the whole pipeline.

A bucket is the head of a **linked list of heap nodes**: one allocation per element, scattered addresses.

That layout isn't laziness: the **bucket API + reference stability** leave no other conforming design.

Hash finds the bucket. **KeyEqual** finds the element. They must agree, or you lose keys.

O(1) means *~1 item per bucket*. Nothing more.

Rules[3]

Order → don't swap a map you iterate for output

Bad Hashes → `hash_combine` over every *identifying* field

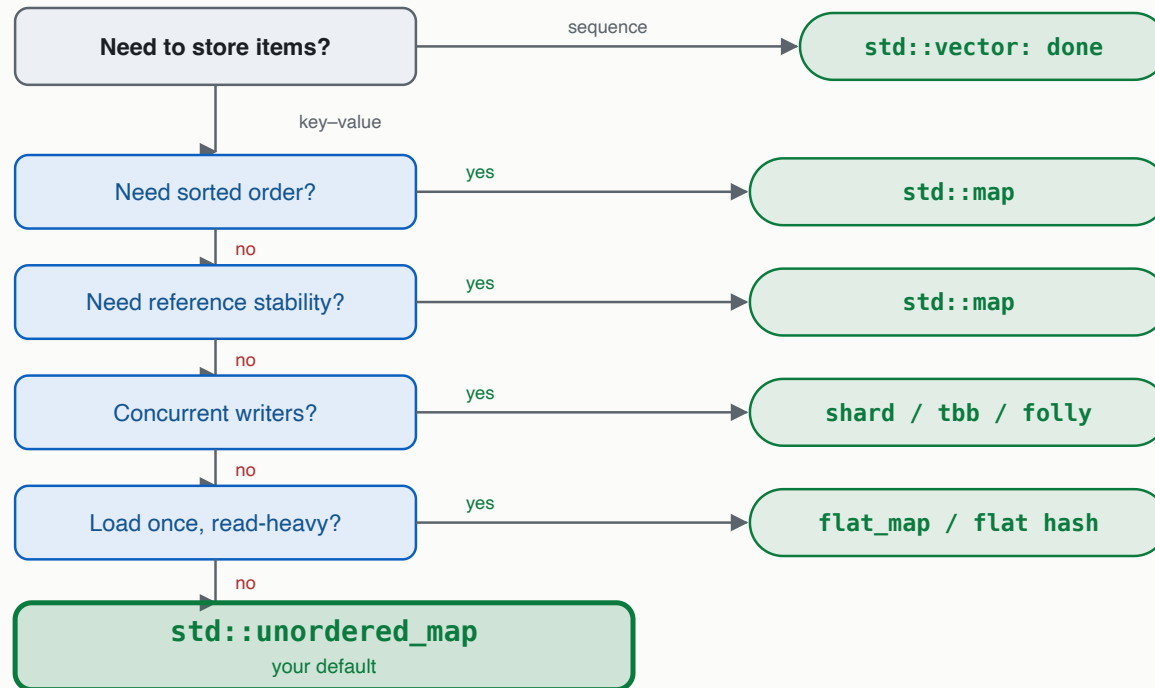
Custom keys → `Hash` and `KeyEqual` must agree

Rehashing → call `reserve( )` when you know the size

...and it is **not thread-safe**. Concurrent read+write is UB.

...and **measure**: the container is rarely your bottleneck.

Your New Decision Tree



Pro tips when you land on `unordered_map`: Custom key? → write a good hash (`hash_combine`). Know the size? → call `reserve()`. Read-heavy + load-once? → consider `flat_map` or a flat hash map.

`src/12_decision_framework_txn.cpp`

Stop using `std::map` "just in case."

Start with `std::unordered_map`, then master its pitfalls: hash all fields, call `reserve()`, know when a flat map wins.

Never swap a map whose *order* you depend on. And **measure**: the container is rarely the bottleneck.

Write faster (and *correct*) modern C++.

Thank You



<packt>

Further Watching & Reading

Matt Kulukundis

Designing a Fast, Efficient, Cache-friendly Hash Table, Step by Step

CppCon 2017: the origin of Google's SwissTable /
`absl::flat_hash_map`

Malte Skarupke

You Can Do Better than `std::unordered_map`: New Improvements to Hash Table Performance

C++Now 2018: Robin Hood hashing, benchmarks, and why
`std::unordered_map` is slow

**Comprehensive
Benchmarks**

martin.ankerl.com: C++ Hash Map Benchmarks 2022

Libraries

`boost::unordered_flat_map` ·
`absl::flat_hash_map` · `ankerl::unordered_dense`